

簡單戰旗遊戲

B11109020 黃鼎鈞

遊戲流程

- 雙方玩家連到伺服器後，初始化資料、分配隊伍與卡牌給雙方。
- 遊戲開始後藍方先攻，執行角色召喚、移動、攻擊
- 其中一方國王死亡結束。

GAMEPANEL

程式邏輯

```
J GamePanel.java > GamePanel
10 public class GamePanel extends JPanel {
47 private ObjectOutputStream in;
28
29 //遊戲執行
30 > public GamePanel(JFrame window, String serverAddress, int port) throws IOException, ClassNotFoundException { ...
55
56 //戰鬥進行UI
57 > private void initializePanels(JFrame window) { ...
83 //戰鬥初始UI
84 > private void configureSplitPanes(JFrame window) { ...
118 //偵測滑鼠點擊
119 > private void configureMouseEvents() { ...
127
128 // 滑鼠點擊事件處理
129 > private void handleMouseClicked(Point click_pos) { ...
138
139 // 滑鼠點擊遊戲狀態處理
140 > private void processClickForTeam(Point click_pos) { ...
173
174 // 處理移動或攻擊
175 > private void processFightAction(Point click_pos) { ...
221
222 // 你的回合選擇角色
223 > private void selectCharacterAt(Point click_pos) { ...
249
250 //不是你的回合選擇角色
251 > private void notYourTurnSelect(Point click_pos) { ...
264
265 //角色繪製
266 > private void drawGameObjects(Graphics g) { ...
290
291
292 //更新卡排版面
293 > private void updateCardPanel() { ...
307
308
309 //更新角色面板
310 > private void updateControlPanel(Warrior warrior) { ...
343
344 //更新敵方角色面板
345 > private void updateEnemyControlPanel(Warrior warrior) { ...
360
361 //清除角色面板
362 > private void clearControlPanel() { ...
368
369 //傳送召喚資訊
370 > private void sendSummonRequest(String warriorName, Point position) { ...
379
380 //傳送傳送Warrior資料
381 > private void sendWarriorUpdate(Warrior warrior) { ...
391
392
393 //從Server接收資料
394 > private void listenToServer() { ...
424
425
426 //回合結束控制
427 > private void endTurn() { ...
451
452
453 //回合結束按鈕
454 > private void updateTurnPanel() { ...
467
468 }
```

SERVER 資料儲存傳輸

```
J GameServer.java > GameServer > switchTurn()
1  import java.awt.Point;
2  import java.io.*;
3  import java.net.*;
4  import java.util.*;
5
6  public class GameServer {
7      private final Map<String, Warrior> warriors = new HashMap<>(); // 所有角色
8      private final Map<Socket, ObjectOutputStream> clients = new HashMap<>(); // 客戶端連接
9      private final Map<Socket, Team> clientTeams = new HashMap<>(); // 客戶端分配的隊伍
10     private Turn currentTurn = Turn.FIRST;
11     public int panelWidth, panelHeight;
12     private boolean hasSummoned = false;
13
14     Run | Debug
15     public static void main(String[] args) { ...
16
17
18
19     public GameServer(int panelWidth, int panelHeight) { ...
20
21
22
23
24     public void startServer(int port) { ...
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49     private void sendTeamAssignment(ObjectOutputStream out, Team team) { ...
50
51
52
53
54
55
56
57
58
59     private void handleClient(Socket clientSocket, ObjectInputStream in) { ...
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87     private void handleSummonRequest(SummonRequest request, Socket clientSocket) { ...
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107     private Warrior findWarriorByNameAndTeam(String name, Team team) { ...
108
109
110
111
112
113
114     private void disconnectClient(Socket clientSocket) { ...
115
116
117
118
119
120
121
122
123
124
125     private void initializeGame() { ...
126
127
128
129
130
131     private void initializeKings() { ...
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146     private void createWarriorsForTeam(Team team) { ...
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167     private void checkVictory() { ...
168
169
170
171
172
173
174
175
176
177
178
179
180
181     private void broadcastMessage(String message) { ...
182
183
184
185
186
187
188
189
190
191
192     private void sendErrorMessage(Socket clientSocket, String message) { ...
193
194
195
196
197
198
199
200
201
202
203     private void broadcastInitialState() { ...
204
205
206
207
208
209
210
211
212
213
214
215
216     private void broadcastGameState() { ...
217
218
219
220
221
222
223
224
225
226
227
228
229     private void switchTurn() { ...
230
231
232
233
234
235 }
236
```

角色類別

```
import java.util.LinkedList;

public class Warrior implements Serializable, Cloneable {
    private static final long serialVersionUID = 1L;

    private int health;
    private int moveRange;
    private String name;
    private LinkedList<Skill> skills;
    private boolean moveControl = true, attackControl = true;
    private boolean selectControl = false;
    private Point size = new Point(x:30, y:50);
    private Rectangle Body;
    private Ellipse2D rangeHitBox;
    private transient BufferedImage image; // 圖片不序列化
    private Team team;
    private Skill selectSkill;
    private State state = State.NULL;
    private type type;

    > public Warrior(int health, int moveRange, BufferedImage img, Team team, type type) { ...
    > public void addSkill(Skill skill) { ...
    > public LinkedList<Skill> getSkills() { ...

    // 移動方法，增加邊界限制
    > public void move(Point position, Rectangle bounds) { ...

    public void attack(Warrior attacked) {
    >     if (attackControl && selectSkill != null) { ...
    >     }

    > public void reduceSkillCooldowns() { ...
    > public int getMoveRange() { ...
    > public void setMoveRange(int range) { ...
    > public int getHealth() { ...
    > public void setHealth(int health) { ...
    > public String getName() { ...
    > public void setName(String name) { ...
    > public boolean getMoveControl() { ...
    > public boolean getAttackControl() { ...
    > public void setMoveControl(boolean control) { ...
    > public void setAttackControl(boolean control) { ...
    > public Point getPosition() { ...
    > public type getType() { ...
    > public void updatePosition(Point position) { ...

    @Override
    > public Warrior clone() { ...
```

啟動

```
import javax.swing.JFrame;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        String serverAddress = "localhost";
        int port = 8080;

        JFrame window = new JFrame(title:"Multiplayer Game");
        window.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        window.setSize(width:1280, height:800);
        window.setResizable(resizable:false);

        try {
            GamePanel gamePanel = new GamePanel(window, serverAddress, port);
            window.add(gamePanel);
        } catch (Exception e) {
            System.err.println("Unable to connect to server: " + e.getMessage());
            e.printStackTrace();
            return;
        }

        window.setVisible(b:true);
    }
}
```